Query Processing
Questions and Solutions

---

1. External Sorting

Q1: External Sort-Merge Runs

Assume (for simplicity) that only one tuple fits in a block and memory holds at most 3 blocks. Show the runs created on each pass of the sort-merge algorithm when applied to sort the following tuples on the first attribute:
(kangaroo, 17), (wallaby, 21), (emu, 1), (wombat, 13), (platypus, 3), (lion, 8), (warthog, 4), (zebra, 11), (meerkat, 6), (hyena, 9), (hornbill, 2), (baboon, 12)

Solution

Label tuples t_1 through t_{12}. Let r_{ij} denote the j-th run of pass i.
Initial Sorted Runs (3 blocks each, sorted alphabetically):

$$r_{11} = \langle t_3(\text{emu}), t_1(\text{kangaroo}), t_2(\text{wallaby})\rangle$$
$$r_{12} = \langle t_6(\text{lion}), t_5(\text{platypus}), t_4(\text{wombat})\rangle$$
$$r_{13} = \langle t_9(\text{meerkat}), t_7(\text{warthog}), t_8(\text{zebra})\rangle$$
$$r_{14} = \langle t_{12}(\text{baboon}), t_{11}(\text{hornbill}), t_{10}(\text{hyena})\rangle$$

After Pass 1 (merge 3 runs at a time, $M - 1 = 2$ way merge):

$$r_{21} = \langle t_3, t_1, t_6, t_9, t_5, t_2, t_7, t_4, t_8\rangle$$
$$r_{22} = \langle t_{12}, t_{11}, t_{10}\rangle$$

After Pass 2 (final sorted run):

$$r_{31} = \langle t_{12}, t_3, t_{11}, t_{10}, t_1, t_6, t_9, t_5, t_2, t_7, t_4, t_8\rangle$$

Alphabetically: baboon, emu, hornbill, hyena, kangaroo, lion, meerkat, platypus, wallaby, warthog, wombat, zebra

2. Join Algorithms

Q2: Join Cost Estimation

Let relations $r_1(A, B, C)$ and $r_2(C, D, E)$ have the following properties: r_1 has 20,000 tuples, r_2 has 45,000 tuples, 25 tuples of r_1 fit on one block, and 30 tuples of r_2 fit on one block. Estimate the number of block transfers and seeks required using each of the following join strategies for $r_1 \bowtie r_2$:

a. Nested-loop join
b. Block nested-loop join
c. Merge join
d. Hash join

Solution

r_1 needs 800 blocks, and r_2 needs 1500 blocks. Let us assume M pages of memory. If $M > 800$, the join can easily be done in $1500 + 800$ disk accesses, using even plain nested-loop join. So we consider only the case where $M < 800$ pages.
Parameters:

$$n_{r_1} = 20,000, \quad b_{r_1} = 800 \qquad n_{r_2} = 45,000, \quad b_{r_2} = 1500$$

a. Nested-Loop Join: Using r_1 as the outer relation:

$$\begin{aligned} \text{Transfers} &= n_{r_1} \times b_{r_2} + b_{r_1} = 20,000 \times 1,500 + 800 = \mathbf{30,000,800} \\ \text{Seeks} &= n_{r_1} + b_{r_1} = 20,000 + 800 = \mathbf{20,800} \end{aligned}$$

Using r_2 as the outer relation:

$$\begin{aligned} \text{Transfers} &= n_{r_2} \times b_{r_1} + b_{r_2} = 45,000 \times 800 + 1,500 = \mathbf{36,001,500} \\ \text{Seeks} &= n_{r_2} + b_{r_2} = 45,000 + 1,500 = \mathbf{46,500} \end{aligned}$$

b. Block Nested-Loop Join: Using r_1 as the outer relation:

$$\begin{aligned} \text{Transfers} &= \left\lceil \frac{b_{r_1}}{M-2} \right\rceil \times b_{r_2} + b_{r_1} \\ \text{Seeks} &= 2 \times \left\lceil \frac{b_{r_1}}{M-2} \right\rceil \end{aligned}$$

c. Merge Join: If neither relation is sorted, we first sort both using external merge sort:

- **Sorting** r_1:

$$\begin{aligned} \text{Transfers}_{r_1} &= b_{r_1} \left(2 \left\lceil \log_{M-1}(b_{r_1}/M) \right\rceil + 1\right) \\ \text{Seeks}_{r_1} &= 2\lceil b_{r_1}/M \rceil + \frac{b_{r_1}}{b_b} \left(2 \left\lceil \log_{M-1}(b_{r_1}/M) \right\rceil - 1\right) \end{aligned}$$

- **Sorting** r_2:

$$\text{Transfers}_{r_2} = b_{r_2}\left(2\left\lceil \log_{M-1}(b_{r_2}/M)\right\rceil + 1\right)$$
$$\text{Seeks}_{r_2} = 2\lceil b_{r_2}/M\rceil + \frac{b_{r_2}}{b_b}\left(2\left\lceil \log_{M-1}(b_{r_2}/M)\right\rceil - 1\right)$$

- **Merge Phase**:

$$\text{Transfers}_{merge} = b_{r_1} + b_{r_2} = 800 + 1,500 = \mathbf{2,300}$$
$$\text{Seeks}_{merge} = \lceil b_{r_1}/b_b\rceil + \lceil b_{r_2}/b_b\rceil$$

d. Hash Join:

- **Partitioning Phase**:

$$\text{Transfers}_{part} = 2(b_{r_1} + b_{r_2}) = 2(800 + 1,500) = \mathbf{4,600}$$
$$\text{Seeks}_{part} = 2(\lceil b_{r_1}/b_b\rceil + \lceil b_{r_2}/b_b\rceil)$$

- **Build and Probe Phase**:

$$\text{Transfers}_{join} = b_{r_1} + b_{r_2} = 800 + 1,500 = \mathbf{2,300}$$
$$\text{Seeks}_{join} = 2 \times n_h \quad (\text{where } n_h = \lceil b_{r_1}/M\rceil)$$

- **Total Transfers**: $3(b_{r_1} + b_{r_2}) = 3(800 + 1,500) = \mathbf{6,900}$

Q3: Optimal Join with Infinite Memory

Let r and s be relations with no indices, and assume they are not sorted. Assuming infinite memory, what is the lowest-cost way to compute $r \bowtie s$? What memory is required?

Solution

Strategy: Load the smaller relation entirely into memory, then stream the larger relation.

Algorithm:

1. Load smaller relation (say s) into memory
2. Build hash index on s
3. Read larger relation r block by block
4. For each r tuple, probe hash index on s

Cost:

$$\text{Block Transfers} = b_r + b_s$$

Memory Required:

$$M = \min(b_r, b_s) + 2 \text{ blocks}$$

(One for the smaller relation, one for input buffer, one for output buffer)

Q4: Secondary Index Inefficiency

The indexed nested-loop join can be inefficient if the index is a secondary index and there are multiple tuples with the same value for the join attributes. Why? Describe a way using sorting to reduce the cost.

Solution

Why Inefficient:
With a secondary index, matching tuples may be scattered across many blocks. If multiple inner tuples match each outer tuple, we may need to access many different blocks per outer tuple — lots of random I/O.
Sorting-Based Optimization:

1. Join outer relation with secondary index **leaf entries only** (not actual tuples)
2. Result: (outer tuple, inner tuple RID) pairs
3. **Sort** result by inner tuple RIDs (physical addresses)
4. Fetch inner tuples in **physical storage order** (sequential I/O)
5. Complete the join

When Better Than Hybrid Merge Join:
Hybrid merge join requires outer relation to be sorted. This algorithm doesn't. If outer relation is much larger than inner, the index lookup cost is less than sorting cost, making this more efficient.

Q5: Buffer Size Trade-off

What is the effect on the cost of merging runs if the number of buffer blocks per run is increased while overall memory available remains fixed?

Solution

Trade-off:

Larger b_b (buffer per run)	**Effect**
Pro:	Fewer seeks (more sequential I/O)
Con:	Fewer runs can merge per pass
Con:	May need more merge passes

Optimal Strategy:
Choose b_b that minimizes total cost:

$$\text{Total Cost} = \text{Transfer Cost} + \text{Seek Cost}$$

With $t_S = 4$ ms and $t_T = 0.1$ ms, seeks are $40\times$ more expensive than transfers, so larger b_b is usually beneficial until it causes extra passes.

3. Pipelining

Q6: Iterator for Indexed Nested-Loop Join

Write pseudocode for an iterator that implements indexed nested-loop join, where the outer relation is pipelined. Define `open()`, `next()`, and `close()`. Show what state information must be maintained.

Solution

State Variables:

- `outer`: Iterator for pipelined outer relation
- `inner`: Iterator for index lookup on inner relation
- `t_r`: Current outer tuple
- `done_r`: Boolean flag for end of outer relation

```
IndexedNLJoin::open():
    outer.open()
    inner.open()
    done_r = false
    if outer.next() returns tuple:
        t_r = that tuple
    else:
        done_r = true

IndexedNLJoin::next():
    while not done_r:
        // Try to get next matching inner tuple
        if inner.next(t_r[JoinAttrs]) returns tuple t_s:
            return t_r JOIN t_s
        else:
            // No more matches for current t_r
            if outer.next() returns tuple:
                t_r = that tuple
                inner.rewind()  // Reset for new t_r
            else:
                done_r = true
    return NULL  // No more results

IndexedNLJoin::close():
    outer.close()
    inner.close()
```

4. Extended Operations

Q7: Semijoin and Anti-Semijoin Implementation

Describe how to implement the following using sorting and hashing:

a. **Semijoin** ($r \ltimes_\theta s$): Output r tuples that have at least one match in s
b. **Anti-semijoin** ($r \bar{\ltimes}_\theta s$): Output r tuples that have NO match in s

Solution

a. Semijoin ($r \ltimes s$):
Using Sorting:

1. Sort both r and s on join attributes
2. Merge scan: when r tuple matches any s tuple, output the r tuple

Using Hashing:

1. Build hash index on s (join attributes)
2. For each r tuple, probe index
3. If match found, output r tuple

b. Anti-Semijoin ($r \bar{\ltimes} s$):
Using Sorting:

1. Sort both r and s on join attributes
2. Merge scan: output r tuple only if NO matching s tuple exists

Using Hashing:

1. Build hash index on s (join attributes)
2. For each r tuple, probe index
3. If NO match found, output r tuple

Alternative (for Anti-Semijoin):

1. Build hash index on r
2. Probe with each s tuple; delete matching r tuples
3. Output remaining r tuples in index

Q8: Top-K Queries and Pipelining

Suppose a query retrieves only the first K results and terminates. Which is better: demand-driven or producer-driven pipelining? Explain.

Solution

Answer: Demand-driven pipelining is better.
Reasoning:

- **Demand-driven:** Only generates exactly K results. The consumer calls

`next()` exactly K times, then stops. No wasted computation.

- **Producer-driven:** Operators eagerly generate tuples and push them into buffers. May generate many more than K tuples before the query terminates. Wasted computation and I/O.

Example: For `SELECT * FROM large_table LIMIT 10`:

- Demand-driven: Generates exactly 10 tuples
- Producer-driven: May fill buffers with thousands of tuples

Q9: Column-Oriented Storage and Joins

Explain why nested-loops join would work poorly on a column-oriented database. Describe a better algorithm.

Solution

Problem with Nested-Loops Join:
Column stores store each attribute separately. Nested-loops join requires assembling full tuples before comparing them. For each tuple:

- Must fetch each attribute from a different disk location
- Tuple assembly is expensive (random I/O per attribute)
- Many assembled tuples are discarded (don't satisfy join condition)
- Wasteful to assemble tuples that won't match

Better Algorithm:

1. **Join on columns only:** Access only the join attribute columns
2. Use hash join, merge join, or indexed nested-loops on these columns
3. Result: List of matching (record_id_r, record_id_s) pairs
4. **Late materialization:**
   - Sort result by record_id of one relation
   - Fetch needed attributes for that relation (sequential)
   - Re-sort by record_id of other relation
   - Fetch needed attributes for other relation

Benefit: Only assemble tuples that are actually in the join result.

Q10: When is Column-Oriented Storage Beneficial?

For each query, indicate if column-oriented storage is beneficial:

a. Fetch ID, name, and dept_name of student with ID 12345
b. Group takes by year and course_id, count students per group

Solution

a. Point Query (ID = 12345):
Column store is NOT beneficial.

- Need to fetch 3 attributes for 1 tuple
- Column store: 3 separate I/O operations (one per column)
- Row store: 1 I/O fetches the entire tuple
- Row store is faster for point queries fetching multiple attributes

b. Aggregation Query:
Column store IS beneficial.

- Only need `year` and `course_id` columns
- Don't need `grade`, `semester`, or other attributes
- Column store: Read only 2 columns (compact, sequential)
- Row store: Read entire tuples (wasteful)
- Column store also benefits from better compression

5. Query Optimization

Q11: Efficient Relational Algebra Expression

Consider the bank database where the primary keys are underlined:

- `branch(branch_name, branch_city, assets)`
- `customer (customer_name, customer_street, customer_city)`
- `loan (loan_number, branch_name, amount)`
- `borrower (customer_name, loan_number)`
- `account (account_number, branch_name, balance)`
- `depositor (customer_name, account_number)`

And the following SQL query:

```
select T.branch_name
from branch T, branch S
where T.assets > S.assets and S.branch_city = 'Brooklyn'
```

Write an efficient relational-algebra expression that is equivalent to this query. Justify your choice.

Solution

Efficient Expression:

$$\Pi_{T.branch\_name}\left(\Pi_{branch\_name,assets}(\rho_T(branch)) \bowtie_{T.assets>S.assets} \Pi_{assets}(\sigma_{branch\_city=\text{'Brooklyn'}}(\rho_S(branch)))\right)$$

Justification: This expression performs the theta join on the smallest amount of data possible. It does this by:

1. **Restricting the right-hand side**: The selection $\sigma_{branch\_city=\text{'Brooklyn'}}$ is applied early to reduce the number of tuples in S.
2. **Eliminating unneeded attributes**: Projections (Π) remove attributes like

`branch_city` from the operands before the join, reducing the size of tuples and thus the total data processed by the theta join.

Q12: Handling Negation with B$^+$-Tree Index

Given a B$^+$-tree index on `branch_city` for relation `branch`, how would you handle:

a. $\sigma_{\neg(branch\_city<\text{"Brooklyn"})}(branch)$
b. $\sigma_{\neg(branch\_city=\text{"Brooklyn"})}(branch)$
c. $\sigma_{\neg(branch\_city<\text{"Brooklyn"}\vee assets<5000)}(branch)$

Solution

a. $\neg(branch\_city < \textbf{"Brooklyn"}) \equiv branch\_city \geq \textbf{"Brooklyn"}$
Use index:

1. Locate first tuple where $branch\_city =$ "Brooklyn"
2. Follow pointer chain to end, retrieving all tuples

b. $\neg(branch\_city = \textbf{"Brooklyn"})$
Index NOT useful. Must scan entire file sequentially and select tuples where $branch\_city \neq$ "Brooklyn".
c. $\neg(branch\_city < \textbf{"Brooklyn"} \vee assets < 5000)$
Apply De Morgan's law:

$$\equiv branch\_city \geq \text{"Brooklyn"} \wedge assets \geq 5000$$

Use index for first condition, then apply second condition:

1. Use index to find tuples with $branch\_city \geq$ "Brooklyn"
2. For each tuple, check if $assets \geq 5000$

6. Cost Analysis

Q13: Sorting Cost with HDD vs Flash

Sort a 40 GB relation with 4 KB blocks using 40 MB memory. Seek time = 5 ms, transfer rate = 40 MB/s.

a. Find the cost with $b_b = 1$ and $b_b = 100$
b. How many merge passes are required in each case?
c. Recalculate for flash storage (latency = 20 μs, transfer = 400 MB/s)

Solution

Given:

- $b_r = 40$ GB/4 KB $= 10,000,000$ blocks

- $M = 40$ MB/4 KB $= 10,000$ blocks
- $t_S = 5$ ms (HDD), $t_T = 4$ KB/40 MB/s $= 0.1$ ms

a. Cost with HDD:
Number of initial runs: $\lceil 10,000,000/10,000 \rceil = 1,000$
With $b_b = 1$: Can merge $M - 1 = 9,999$ runs per pass $\Rightarrow$ **1 merge pass**
With $b_b = 100$: Can merge $\lfloor M/b_b \rfloor - 1 = 99$ runs per pass $\Rightarrow$ **2 merge passes**
Block Transfers: $b_r(2P + 1)$ where $P =$ passes

- $b_b = 1$: $10^7 \times 3 = 30 \times 10^6$ transfers $= 30 \times 10^6 \times 0.1$ ms $=$ **3000 sec**
- $b_b = 100$: $10^7 \times 5 = 50 \times 10^6$ transfers $=$ **5000 sec**

But $b_b = 100$ has 100× fewer seeks, so total time differs based on seek overhead.
c. Flash Storage:
$t_S = 0.02$ ms, $t_T = 4$ KB/400 MB/s $= 0.01$ ms
Seeks are 250× cheaper on flash vs HDD! Transfer is 10× faster.
With flash, total time is dominated by transfers, so $b_b = 1$ is better (fewer passes).

Q14: Hash Join Extension for Outer Joins

Describe how to extend the hash-join algorithm to compute:

a. Natural left outer join
b. Natural right outer join
c. Natural full outer join

Solution

Key Idea: Track which tuples have been matched.
a. Left Outer Join ($r \bowtie\!\!\!\!\!\!\bowtie s$):

1. Build hash index on s (build relation)
2. For each tuple t_r in r (probe relation):
    - Probe hash index for matches
    - If match found: output joined tuples
    - If NO match: output t_r padded with NULLs

b. Right Outer Join ($r \bowtie\!\!\!\!\!\!\bowtie s$):

1. Build hash index on s; add a **matched flag** to each entry
2. For each t_r in r: probe and mark matched s tuples
3. After probing: scan hash index for unmatched s tuples
4. Output unmatched s tuples padded with NULLs

c. Full Outer Join ($r \bowtie\!\!\!\!\!\!\bowtie s$): Combine both strategies:

1. Build hash index on s with matched flags
2. For each t_r in r:
    - If match: output join, mark s tuples as matched
    - If no match: output t_r with NULLs
3. Scan hash index: output unmatched s tuples with NULLs

Q15: Computing Multiple Aggregations with Single Sort

Suppose you need to compute both:

$$_{A}\gamma_{sum(C)}(r) \quad \text{and} \quad _{A,B}\gamma_{sum(C)}(r)$$

Describe how to compute these together using a single sorting of r.

Solution

Strategy: Sort once on (A, B), then compute both aggregates in one scan.
Algorithm:

1. Sort r on (A, B)
2. Initialize: $sum\_A = 0$, $sum\_AB = 0$, $current\_A$, $current\_B$
3. Scan sorted relation:
   - If (A, B) changes: output $(current\_A, current\_B, sum\_AB)$, reset $sum\_AB = 0$
   - If A changes: output $(current\_A, sum\_A)$, reset $sum\_A = 0$
   - Add C value to both sum_A and sum_AB
4. Output final group values

Why This Works:

- Sorting on (A, B) groups by A first, then by B within each A
- All tuples with same A are contiguous
- Within same A, tuples with same (A, B) are contiguous
- One pass computes both aggregates

Cost: Sorting cost + single scan = much cheaper than sorting twice.

Q16: Instruction Cache and Pipelining

Modern CPUs have instruction caches. Function calls cause cache misses.

a. Why does producer-driven pipelining have better instruction cache hit rate?
b. How can demand-driven pipelining be modified to improve cache hits?

Solution

a. Producer-Driven Advantage:

- Each operator runs in a loop, generating many tuples
- Same instructions execute repeatedly ⇒ stay in cache
- No function call overhead between tuples
- Context switches only when buffer is full/empty

Demand-Driven Problem:

- Each `next()` call switches to a different operator's code
- Frequent function calls ⇒ frequent instruction cache misses

- For each tuple: call chain through multiple operators

b. Improving Demand-Driven:
Batch `next()` calls: Return multiple tuples per call.

```
// Instead of:
tuple = operator.next()  // 1 tuple

// Use:
tuples[] = operator.next_batch(100)  // 100 tuples
```

Benefits:

- Operator executes loop to generate batch ⇒ instructions stay in cache
- Amortizes function call overhead over many tuples
- Similar cache behavior to producer-driven

Q17: Relational Division Implementation

Design sort-based and hash-based algorithms for computing the relational division operation $r \div s$ where $r(T, S)$ and $s(S)$.

Solution

Recall: $r \div s$ returns T values from r that are associated with ALL S values in s.
Sort-Based Algorithm:

1. Sort s on S
2. Sort r on (T, S)
3. Scan r: for each distinct T value:
    - Compare S values in r with all of s (merge-like scan)
    - If every s tuple appears with this T value, output T

Cost: Sorting cost + $|r| + N \times |s|$ where N = distinct T values.
Hash-Based Algorithm:

1. Partition r on T attributes
2. For each partition:
    - Build hash index on (T, S)
    - Collect all distinct T values in separate hash table
    - For each distinct T value v_T:
        - For each s value v_S in s: probe for (v_T, v_S)
        - If any probe fails, discard v_T
        - If all probes succeed, output v_T

Q18: Hybrid Merge Join with Two Unsorted Relations

Design a variant of hybrid merge-join for the case where both relations are not physically sorted, but both have a sorted secondary index on the join attributes. Estimate the number of block transfers and seeks required.

Solution

Problem: Secondary indices give sorted access but tuples are scattered on disk. A naive join would require one random I/O per matched tuple.
Algorithm:

1. **Index Merge**: Scan both secondary indices in sorted order and perform a merge-join on (key, RID) entries.
2. **Materialization**: Store the resulting (RID_r, RID_s) pairs in a temporary relation T_{join}.
3. **First Sort & Fetch**:
   - Sort T_{join} by RID_r (physical address of r tuples).
   - Scan sorted T_{join} and r in parallel; fetch attributes of r and attach to RIDs.
4. **Second Sort & Fetch**:
   - Sort result by RID_s (physical address of s tuples).
   - Scan result and s in parallel; fetch attributes of s and output joined tuples.

Cost Estimation:

- **Transfers**:

$$\begin{aligned} \text{Transfers} &= (b_{idx\_r} + b_{idx\_s}) && \text{(Index leaf scans)} \\ &+ \text{sorting cost for } T_{join} && \text{(Depends on result size)} \\ &+ (b_r + b_s) && \text{(Sequential tuple fetches)} \end{aligned}$$

- **Seeks**:

$$\begin{aligned} \text{Seeks} &\approx (\lceil b_{idx\_r}/b_b \rceil + \lceil b_{idx\_s}/b_b \rceil) && \text{(Initial scans)} \\ &+ \text{sorting seeks for } T_{join} \\ &+ (\lceil b_r/b_b \rceil + \lceil b_s/b_b \rceil) && \text{(Final fetches)} \end{aligned}$$

Advantage: Avoids random I/O for tuple fetches.

Q19: Finding Documents with k of n Keywords

You want to find documents containing at least k of a given set of n keywords. You have a keyword index giving sorted document IDs for each keyword. Give an efficient algorithm.

Solution

Algorithm:

1. Initialize empty list L of (doc_id, count) records
2. For each keyword w in the set of n keywords:
   - Get sorted document list D_w from index
   - Merge D_w with L:
     - If doc_id already in L: increment its count
     - Otherwise: add (doc_id, 1) to L
3. Output all records in L where count $\geq k$

Optimization: Keep L sorted by doc_id for efficient merging.
Complexity:

$$O\left(n \times \sum_{i=1}^{n} |D_{w_i}|\right)$$

Each merge is linear in the sum of list lengths.
Alternative (for large n):

1. Use min-heap with one pointer per keyword list
2. Extract minimum doc_id, count occurrences
3. Output if count $\geq k$

Q20: Pipelining Sort-Merge with Shared Memory

When both inputs to merge join come from sort operations:

a. How is the sort operator broken into sub-operators for pipelining?
b. What is the effect of sharing memory between two merge operations?

Solution

a. Sort Operator Sub-Operators:
External sort-merge has two phases that can be modeled separately:

1. **Run Generation (blocking input, blocking output):**
   - Can pipeline *from* its input
   - Blocks until all runs are generated
2. **Merge Phase (blocking input, pipelined output):**
   - Waits for run generation to complete
   - Can pipeline *to* its consumer (merge-join)

```
[Input] --pipelined--> [Run Gen] --blocking--> [Merge] --pipelined--> [Join]
```

b. Shared Memory Effect:
If memory M is shared between two sort-merge operations:

- Each sort gets $M/2$ memory
- Smaller memory $\Rightarrow$ more initial runs per relation

- More runs ⇒ potentially more merge passes
- Each merge pass: $O(b_r)$ I/O

Trade-off:

$$\text{Passes with } M/2 = \lceil \log_{M/2-1}(b_r/(M/2)) \rceil$$

May need 1-2 extra passes compared to having full M memory.
Mitigation: Execute sorts sequentially if possible, giving each full memory.

Q21: Concept Hierarchy Indexing

Suggest how a document containing a word (such as "leopard") can be indexed such that it is efficiently retrieved by queries using a more general concept (such as "carnivore" or "mammal"). You can assume that the concept hierarchy is not very deep, so each concept has only a few generalizations (a concept can, however, have a large number of specializations). You can also assume that you are provided with a function that returns the concept for each word in a document. Also suggest how a query using a specialized concept can retrieve documents using a more general concept.

Solution

1. Retrieving specialized documents with general queries: We can use **indexing-time expansion** (or generalization indexing). For each word w in the document:

1. Use the provided function to find the concept for w.
2. Traverse the concept hierarchy upwards to find all generalizations of that concept.
3. Index the document under the word itself AND under all its more general concepts.

Example: A document with "leopard" is indexed under "leopard", "carnivore", and "mammal". Since the hierarchy is shallow, the number of extra index entries per word is small and manageable.
2. Retrieving general documents with specialized queries: If a query uses a specialized concept (e.g., "leopard"), but we want to retrieve documents that mention more general terms (e.g., "carnivore"), we use **query expansion**:

- When the user enters a specialized query term S, look up its generalizations in the hierarchy.
- Automatically add these generalizations to the query (e.g., using an `OR` operator or by assigning them lower weights).

Benefit: This allows a user searching for "leopard" to also find relevant general documents about "carnivores" if no specialized matches are found.

Q22: Hybrid Hash-Join Sub-operators and Pipelining

Explain how to split the hybrid hash-join operator into sub-operators to model pipelining. Also explain how this split is different from the split for a hash-join operator.

Solution

Hybrid hash-join can be split into three sub-operators:

1. **Hybrid-Hash-Partition-Build**: Reads the build relation r, partitions it into $H_{r0}, H_{r1}, \ldots, H_{rn}$, keeps H_{r0} in memory, and writes other partitions to disk. This is a *blocking* sub-operator on its input r.

2. **Hybrid-Hash-Partition-Probe**: Reads the probe relation s and partitions it. Crucially, as tuples belonging to H_{s0} are identified, they are **immediately joined** with the in-memory H_{r0} and produced as output. Other partitions are written to disk. This sub-operator is *pipelined* on its input s and produces pipelined output.

3. **Hybrid-Hash-Join-Disk-Partitions**: Reads and joins the remaining pairs of partitions from disk ($H_{r1} \bowtie H_{s1}, \ldots, H_{rn} \bowtie H_{sn}$). This is a *blocking* sub-operator.

Difference from standard Hash-Join: In a standard hash-join, the probe relation partitioning is a completely blocking phase; no results can be produced until the entire probe relation has been partitioned and written to disk. In hybrid hash-join, joined results for the first partition are produced **concurrently** with the partitioning of the probe relation, improving response time and enabling earlier pipelining to higher-level operators.

7. Quick Reference

Algorithm	Block Transfers	Notes
Nested-Loop	$n_r \times b_s + b_r$	Very expensive
Block Nested-Loop	$b_r \times b_s + b_r$	Much better
Merge Join (sorted)	$b_r + b_s$	Best if pre-sorted
Hash Join	$3(b_r + b_s)$	Good for large unsorted